



Application de Paiement Etudiant

Documentation Scientifique et Technique

Auteur: Patrice Beausoleil

Version 2.0.0

L3 Informatique - Programmation Mobile 2026

BKN - Application de Paiement Étudiant

*Application mobile de paiement nouvelle génération développée en Flutter,
avec un backend FastAPI connecté à une base de données PostgreSQL hébergée sur Supabase.*

Auteur : Patrice Beausoleil

Version : 2.0.0

Branche GitHub : `beausoleil`

Cours : L3 Informatique - Programmation Mobile 2026

Table des matières

1. Présentation du projet
 2. Fonctionnalités
 3. Stack technique
 4. Architecture du projet
 5. Prérequis
 6. Installation du backend
 7. Installation de l'application mobile
 8. Tests en local avec Docker
 9. API - Endpoints disponibles
 10. Base de données
 11. Infrastructure & Déploiement
 12. Dépendances détaillées
-

1. Présentation du projet

BKN (prononcé "BéKéN") est une application mobile de paiement fictive développée dans le cadre du cours de programmation mobile de L3.

Elle simule un portefeuille numérique complet dans lequel les étudiants peuvent gérer un solde, acheter des cryptomonnaies, payer par QR code et discuter avec un assistant virtuel.

- Contenu du projet :

- * Application mobile Flutter (Android)
 - * API REST backend avec FastAPI (Python 3.11)
 - * Base de données PostgreSQL hébergée sur Supabase
 - * Tests en local avec Docker Desktop (API + BDD conteneurisées)
 - * Authentification sécurisée (bcrypt + JWT)
 - * Upload de photos de profil
 - * Transactions BKN (1 BKN = 1 EUR)
 - * Achat/vente de cryptomonnaies
 - * Paiement par QR code (génération + scan)
 - * Chatbot intégré "Félicité"
-

2. Fonctionnalités

Authentification

- Connexion & Inscription :

- * Page de connexion sécurisée (email + mot de passe)
- * Inscription avec bonus de bienvenue de 100 BKN
- * Mot de passe haché avec l'algorithme bcrypt
- * Session persistante via flutter_secure_storage
- * Changement de mot de passe depuis le profil

Profil utilisateur

- Gestion du compte :

- * Affichage des informations personnelles (nom, prénom, pseudo, email, téléphone)
- * Modification du profil en temps réel
- * Upload de photo de profil (multipart vers le backend, stockée dans /avatars/)
- * Niveau de vérification (Niveau 1 / Niveau 2)

- Paramètres de sécurité :

- * Activation/désactivation de la biométrie (toggle)
- * Gestion des notifications push
- * Authentification à deux facteurs (2FA - toggle UI)

- * Liste des sessions actives (appareils connectés)
- * Déconnexion d'une session à distance

Portefeuille BKN

- Opérations financières :

- * Solde en temps réel (rafraîchi depuis l'API)
- * Achat de BKN par carte bancaire (simulé via Stripe)
- * Vente de BKN avec mise à jour instantanée
- * Transfert entre utilisateurs (par pseudo @user, email ou ID)
- * Historique complet des transactions

Cryptomonnaies

- Cryptos supportées :

- * Bitcoin (BTC) - 45 000 EUR
- * Ethereum (ETH) - 2 800 EUR
- * Solana (SOL) - 98 EUR
- * Cardano (ADA) - 0,45 EUR
- * Polkadot (DOT) - 6,50 EUR
- * Avalanche (AVAX) - 35 EUR

- Fonctionnement :

- * Achat de crypto en dépensant des BKN
- * Vente de crypto pour récupérer des BKN
- * Prix fixés au moment de la transaction (price_at_transaction)
- * Historique des transactions crypto par utilisateur

Paiement QR Code

- Fonctionnement :

- * Génération d'un QR code personnel (encodage de l'ID utilisateur)
- * Scan de QR code via la caméra pour initier un transfert
- * Transaction instantanée après scan
- * Utilise qr_flutter (génération) et mobile_scanner (lecture)

Chatbot Félicité

- Assistant virtuel :

- * Interface de chat intégrée à l'application
- * Répond aux questions fréquentes (solde, transactions, aide)
- * Accessible depuis la barre de navigation principale

Analyses

- Tableau de bord :

- * Écran d'analytiques avec visualisation du solde
 - * Graphiques des dépenses et revenus
 - * Historique filtrable des transactions
-

3. Stack technique

Application mobile

- Technologies Flutter :

- * Flutter ? 3.0.0 - Framework mobile cross-platform
- * Dart ? 3.0.0 - Langage de programmation
- * Provider ^6.1.2 - Gestion d'état (State Management)
- * http ^1.3.0 - Requêtes HTTP vers l'API REST
- * flutter_secure_storage - Stockage sécurisé des tokens de session
- * qr_flutter ^4.1.0 - Génération de QR codes
- * mobile_scanner ^6.0.6 - Lecture de QR codes via caméra
- * image_picker ^1.1.2 - Sélection et upload de photo de profil
- * bonsoir ^6.0.2 - Découverte réseau mDNS (auto-détection IP)
- * google_fonts ^6.2.1 - Typographie (SF Pro / Inter)
- * flutter_animate ^4.5.2 - Animations fluides des composants
- * shared_preferences - Persistance locale (clé/valeur)
- * intl ^0.19.0 - Formatage des dates et montants
- * shimmer ^3.0.0 - Effet de chargement skeleton

Backend

- Technologies Python :

- * Python 3.11 - Langage backend

- * FastAPI 0.110.0 - Framework API REST asynchrone (30+ endpoints)
- * Uvicorn 0.27.1 - Serveur ASGI
- * psycopg2-binary 2.9.9 - Connecteur PostgreSQL natif
- * Pydantic 2.6.3 - Validation automatique des données / schémas
- * passlib + bcrypt - Hachage sécurisé des mots de passe
- * python-jose 3.5.0 - Génération et vérification de tokens JWT
- * zeroconf 0.148.0 - Publication mDNS (découverte réseau locale)
- * sendgrid 6.10.1 - Envoi d'emails (reset mot de passe)
- * tenacity 8.2.2 - Retry automatique sur la connexion à la BDD

Infrastructure

- Évolution de l'infrastructure (du développement à la production) :

- * Étape 1 - Docker Desktop : Développement local (API + PostgreSQL dans Docker)
- * Étape 2 - Render.com : 1ère BDD en ligne (PostgreSQL Render, alternative)
- * Étape 3 - Supabase : Migration finale de la BDD (plus stable, SSL, pooler)

- Services utilisés :

- * Docker Desktop - Tests en local (PostgreSQL 15 + FastAPI conteneurisés)
- * Render.com - Alternative BDD en ligne (PostgreSQL gratuit Render)
- * Supabase - Base de données finale (PostgreSQL AWS Oregon, port 6543)
- * SendGrid - API email pour les resets de mot de passe

4. Architecture du projet

```
mobileapp-2026/
|
+-- README.md                <- Ce fichier
|
+-- infrastructure/          <- Documentation et fichiers d'infra
|   +-- README.md            <- Schéma architecture, tâches, sécurité
|   +-- database/
|       +-- schema.sql        <- Création des 5 tables PostgreSQL
|       +-- seed.sql          <- Données de test (4 users, transactions)
|   +-- deploy/
|       +-- render.md         <- Historique Render.com (1ère BDD en ligne)
|       +-- supabase.md       <- Guide Supabase (solution finale actuelle)
```

```

|   +-- env/
|       +-- .env.example           <- Template des variables d'environnement
|
+-- Backend/                      <- API FastAPI
|   +-- server.py                 <- 1400+ lignes, 35+ endpoints REST
|   +-- requirements.txt          <- Dépendances Python
|   +-- Dockerfile                <- Image Python 3.11-slim
|   +-- docker-compose.yml        <- Orchestration API + PostgreSQL local
|   +-- docker-start.bat          <- Script de démarrage Windows
|   +-- docker-stop-all.bat      <- Script d'arrêt Windows
|   +-- avatars/                 <- Photos de profil uploadées
|
+-- app_bkn/                      <- Application Flutter
    +-- pubspec.yaml              <- Dépendances Flutter
    +-- lib/
        +-- main.dart             <- Point d'entrée de l'application
        +-- models/               <- Modèles de données
            |   +-- user.dart
            |   +-- transaction.dart
            |   +-- wallet.dart
            |   +-- crypto.dart
        +-- providers/             <- State management (Provider pattern)
            |   +-- user_provider.dart
            |   +-- transaction_provider.dart
            |   +-- crypto_provider.dart
        +-- screens/               <- 15 écrans de l'application
            |   +-- splash_screen.dart
            |   +-- login_screen.dart
            |   +-- register_screen.dart
            |   +-- home_screen.dart
            |   +-- profile_screen.dart
            |   +-- edit_profile_screen.dart
            |   +-- security_screen.dart
            |   +-- buy_screen.dart
            |   +-- sell_screen.dart
            |   +-- transfer_screen.dart
            |   +-- crypto_screen.dart
            |   +-- history_screen.dart
            |   +-- analytics_screen.dart
            |   +-- scan_screen.dart
            |   +-- qr_receive_screen.dart

```

```

|   +-- chatbot_screen.dart
+-- services/                <- Communication avec l'API
|   +-- api_service.dart    <- Appels HTTP à tous les endpoints
|   +-- api_helper.dart     <- Détection automatique de l'IP serveur (mDNS)
+-- theme/
|   +-- app_theme.dart      <- Système de design (couleurs, typographie)
+-- widgets/                <- Composants réutilisables
    +-- balance_card.dart
    +-- action_grid.dart
    +-- recent_transactions.dart

```

5. Prérequis

- Outils à installer :

- * Flutter SDK (version 3.0 ou supérieure)
 - Inclut le SDK Dart
 - Vérifier l'installation : flutter doctor
- * Android Studio (dernière version)
 - SDK Android
 - Émulateur Android (optionnel)
 - Plugin Flutter + Dart
- * Un smartphone Android physique (recommandé)
 - Modèle de test : Samsung Galaxy S10
 - Mode développeur activé
 - ? Paramètres > À propos du téléphone > Numéro de build (appuyer 7 fois)
 - Débogage USB activé
 - ? Paramètres > Options développeurs > Débogage USB
- * Python 3.11 ou supérieure
- * Docker Desktop - pour les tests en local
 - Windows : Docker Desktop for Windows
 - WSL2 activé recommandé
- * Git

- Comptes en ligne requis :

- * Supabase - Base de données PostgreSQL cloud (<https://supabase.com>)
 - * Render.com - Alternative BDD / hébergeur API (<https://render.com>)
 - * GitHub - Gestion de version (<https://github.com>)
 - * SendGrid - Envoi d'emails reset mot de passe (<https://sendgrid.com>)
-

6. Installation du backend

- Clonage du dépôt :

```
git clone https://github.com/roor-killla/mobileapp-2026.git
cd mobileapp-2026
git checkout beausoleil
```

- Aller dans le dossier Backend :

```
cd Backend
```

- Créer un environnement virtuel Python *(recommandé)* :

```
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # Linux/Mac
```

- Installer les dépendances Python :

```
pip install -r requirements.txt
```

- Configurer les variables d'environnement :

- * Copier le template : infrastructure/env/.env.example ? Backend/.env
- * Remplir les valeurs dans le fichier .env :

```
DB_HOST=aws-0-us-west-2.pooler.supabase.com
DB_PORT=6543
DB_NAME=postgres
DB_USER=postgres.VOTRE_PROJECT_ID
DB_PASSWORD=VOTRE_MOT_DE_PASSE
JWT_SECRET_KEY=une_cle_aleatoire_minimum_32_caracteres
SENDGRID_API_KEY=SG.VOTRE_CLE_API
```

- Lancer le serveur :

```
python server.py
```

- * L'API démarre sur : http://0.0.0.0:8000
 - * Documentation Swagger : http://localhost:8000/docs
 - * La base de données est initialisée automatiquement au premier lancement
 - * Les 4 utilisateurs de test sont créés si la table est vide
-

7. Installation de l'application mobile

- Se placer dans le dossier Flutter :

```
cd app_bkn
```

- Installer les dépendances Flutter :

```
flutter pub get
```

- Configurer l'adresse du serveur :

- * Modifier le fichier lib/services/api_helper.dart
- * Remplacer l'IP par celle de votre machine sur le réseau local :

```
static const String _manualFallbackIp = '192.168.X.X';
```

- * Astuce : Le package bonsoir permet la détection automatique via mDNS.
Si le serveur et le téléphone sont sur le même Wi-Fi, l'IP peut être détectée automatiquement sans configuration manuelle.

- Vérifier la connexion au téléphone :

```
flutter devices
```

- * Le Samsung Galaxy S10 (ou votre appareil) doit apparaître dans la liste
- * Si non détecté : vérifier le câble USB et le débogage USB activé

- Lancer l'application :

```
flutter run
```

8. Tests en local avec Docker

- Démarrer Docker Desktop :

- * Lancer Docker Desktop depuis le menu Démarrer Windows
- * Vérifier que Docker fonctionne :

```
docker --version
docker-compose --version
```

- Lancer l'environnement complet :

```
cd Backend
docker-compose up -d
```

- * Ce qui est lancé :
 - Conteneur bkn_postgres : PostgreSQL 15 Alpine (port 5432)
 - Conteneur bkn_api : FastAPI Python 3.11 (port 8001 ? 8000)
- * Le conteneur bkn_api attend que PostgreSQL soit sain avant de démarrer

- Scripts Windows disponibles :

- * lancer_serveur.bat - Lance uniquement le serveur Python FastAPI (python server.py)
- * docker-start.bat - Lance les conteneurs Docker (docker-compose build + up)
- * docker-stop-all.bat - Arrête tous les conteneurs Docker

- Vérifier les conteneurs :

```
docker-compose ps
docker-compose logs api
```

- Arrêter les conteneurs :

```
docker-compose down          # Arrêt simple
docker-compose down -v       # Arrêt + reset de la base de données
```

9. API - Endpoints disponibles

La documentation complète est auto-générée par FastAPI sur `/docs` (Swagger UI).

- Authentification :

- * POST /login - Connexion utilisateur
- * POST /register - Inscription (crédite 100 BKN de bonus)

- Utilisateurs :

- * GET /users - Liste de tous les utilisateurs actifs
- * GET /user/{id} - Détails d'un user (fonctionne avec ID, pseudo ou email)
- * GET /balance/{id} - Solde BKN d'un utilisateur
- * PUT /user/{id} - Modifier nom, prénom, email, pseudo, téléphone
- * POST /user/{id}/avatar - Upload photo de profil (multipart/form-data)
- * POST /user/change-password - Changer le mot de passe

- Transactions BKN :

- * POST /transfer - Transférer des BKN vers un autre utilisateur
- * POST /buy - Acheter des BKN (crédit du solde)
- * POST /sell - Vendre des BKN (débit du solde)
- * GET /history/{id} - Historique des transactions d'un utilisateur
- * GET /stats - Statistiques globales (total users, transactions, volume)

- Cryptomonnaies :

- * GET /crypto/prices - Prix actuels des 6 cryptos
- * POST /crypto/buy - Acheter une crypto en dépensant des BKN
- * POST /crypto/sell - Vendre une crypto pour récupérer des BKN
- * GET /crypto/balance/{id} - Portefeuille crypto d'un utilisateur
- * GET /crypto/history/{id} - Historique des transactions crypto

- Sécurité & Sessions :

- * GET /user/{id}/settings - Paramètres de sécurité
- * PUT /user/{id}/settings - Modifier biométrie, notifs, 2FA
- * GET /user/{id}/sessions - Sessions actives (appareils connectés)
- * DELETE /user/session/{id} - Terminer une session précise
- * DELETE /user/{id}/sessions - Terminer toutes les sessions

- Utilitaires :

- * GET / - Informations API (nom, version, statut, timestamp)
- * GET /health - Health check (vérifie la connexion à la base de données)

10. Base de données

- Évolution des environnements de base de données :

- * Phase 1 - Docker Desktop (local)
 - PostgreSQL 15 Alpine dans un conteneur Docker
 - Démarrage avec docker-compose up depuis le dossier Backend/
 - Port 5432 exposé, volume persistant bkn_postgres_data
 - Idéal pour tester sans connexion internet
- * Phase 2 - Render.com (1ère base en ligne)
 - Base PostgreSQL gratuite offerte par Render.com
 - Utilisée lors de la première mise en ligne du projet
 - Host : dpg-XXXXXX.oregon-postgres.render.com (port 5432)
 - Limitation : mise en veille après inactivité, données parfois perdues
- * Phase 3 - Supabase (solution finale actuelle)
 - Migration depuis Render vers Supabase pour plus de stabilité
 - PostgreSQL hébergé sur AWS Oregon
 - Connexion via Connection Pooler (port 6543, SSL requis)
 - Backups automatiques, interface web, jamais en veille

- Tables de la base de données :

- * users - Comptes utilisateurs + solde BKN
- * transactions - Historique des opérations (achat, vente, transfert, réception)
- * crypto_transactions - Achats et ventes de cryptomonnaies
- * user_settings - Préférences de sécurité par utilisateur
- * user_sessions - Sessions actives et appareils connectés

- Schéma principal (table users) :

```
CREATE TABLE users (
  id                VARCHAR(50)    PRIMARY KEY,
  email             VARCHAR(255)    UNIQUE NOT NULL,
  nom               VARCHAR(100)    NOT NULL,
  prenom            VARCHAR(100)    NOT NULL,
  pseudo            VARCHAR(50)     UNIQUE NOT NULL,
  phone             VARCHAR(20),
  password_hash     VARCHAR(255)    NOT NULL, -- haché bcrypt
  solde             DECIMAL(15,2)   DEFAULT 1500.00,
  created_at        TIMESTAMP       DEFAULT CURRENT_TIMESTAMP,
  verification_level VARCHAR(50)    DEFAULT 'Niveau 1',
  is_active         BOOLEAN         DEFAULT TRUE,
```

```

    last_login      TIMESTAMP,
    avatar_url      TEXT
);

```

- Utilisateurs de test créés automatiquement :

- * john.doe@email.com / password123 ? 5 000 BKN (Niveau 2)
- * jane.smith@email.com / password123 ? 3 000 BKN (Niveau 1)
- * bob.martin@email.com / password123 ? 2 000 BKN (Niveau 1)
- * alice.wonder@email.com / password123 ? 4 500 BKN (Niveau 2)

11. Infrastructure & Déploiement

Voir le dossier `infrastructure/` pour la documentation complète.

- Évolution de l'infrastructure :

- * Phase locale ? Docker Desktop (API + PostgreSQL tout en local)
- * Phase en ligne ? Render.com (1ère BDD PostgreSQL en ligne, gratuite)
- * Phase finale ? Supabase (migration BDD vers Supabase, plus robuste)

- Architecture actuelle :

```

[Flutter Android] --HTTP--> [FastAPI (server.py)] --SSL--> [PostgreSQL / Supabase]
                                                         aws-0-us-west-2
                                                         port 6543

```

- Fichiers disponibles dans infrastructure/ :

- * infrastructure/database/schema.sql - Création des 5 tables + index
- * infrastructure/database/seed.sql - Données de test (users, transactions, crypto)
- * infrastructure/env/.env.example - Template des variables d'environnement
- * infrastructure/deploy/render.md - Historique + guide Render.com (BDD + API)
- * infrastructure/deploy/supabase.md - Guide migration et config Supabase

- État actuel du projet :

- * Base de données : Supabase (PostgreSQL - AWS Oregon, port 6543)
- * Application : APK Android (installé sur Samsung Galaxy S10)
- * Backend : Lancement via python server.py

12. Dépendances détaillées

- Backend - requirements.txt :

```
fastapi==0.110.0          # Framework API REST asynchrone
uvicorn[standard]==0.27.1 # Serveur ASGI
psycopg2-binary==2.9.9    # Connecteur PostgreSQL natif Python
python-dotenv==1.0.1      # Chargement automatique des fichiers .env
pydantic==2.6.3           # Validation des schémas de données
passlib==1.7.4            # Interface de hachage (wrapper bcrypt)
bcrypt==4.0.1             # Algorithme de hachage des mots de passe
ifaddr==0.2.0             # Listing des interfaces réseau locales
zeroconf==0.148.0         # Découverte réseau mDNS
sendgrid==6.10.1          # API SendGrid pour l'envoi d'emails
python-http-client==3.3.7 # Client HTTP bas niveau (dépendance SendGrid)
```

- Flutter - pubspec.yaml (dépendances principales) :

```
http: ^1.3.0                # Requêtes HTTP (GET, POST, PUT, DELETE)
provider: ^6.1.2            # State management (Pattern Provider)
flutter_animate: ^4.5.2     # Animations de composants
google_fonts: ^6.2.1        # Polices Google (Inter, SF Pro style)
qr_flutter: ^4.1.0          # Génération de QR codes
mobile_scanner: ^6.0.6      # Lecture QR code via caméra
image_picker: ^1.1.2        # Sélection photo depuis galerie/caméra
flutter_secure_storage: ^9.2.4 # Stockage sécurisé (tokens, sessions)
bonsoir: ^6.0.2             # Découverte automatique du serveur (mDNS)
shared_preferences: ^2.3.5   # Stockage local simple (clé/valeur)
intl: ^0.19.0               # Formatage dates et montants (EUR, BKN)
shimmer: ^3.0.0             # Effet squelette pendant le chargement
```

Projet développé dans le cadre de L3 Informatique - Programmation Mobile 2026

Auteur : Patrice Beausoleil | GitHub ? branche beausoleil

Infrastructure BKN - Documentation

Ce dossier contient tous les fichiers et guides liés à l'infrastructure du projet BKN :
déploiement cloud, base de données, Docker et configuration des environnements.

Contenu du dossier infrastructure/

```

infrastructure/
|
+-- README.md                <- Ce fichier (vue d'ensemble)
|
+-- database/
|   +-- schema.sql           <- Création des 5 tables PostgreSQL + index
|   +-- seed.sql             <- Données de test (users, transactions, crypto)
|
+-- deploy/
|   +-- render.md            <- Historique + guide Render.com (BDD en ligne
alternative)
|   +-- supabase.md          <- Guide configuration Supabase (solution finale)
|
+-- env/
|   +-- .env.example         <- Template des variables d'environnement

```

Architecture actuelle

```

+-----+
|           Application Flutter (Android)           |
|           lib/services/api_helper.dart             |
|           Découverte auto du serveur via mDNS (bonsoir) |
+-----+

```

```

| HTTP REST (JSON)
v
+-----+
| Backend FastAPI - Python 3.11 |
| server.py - 1400+ lignes - 35+ endpoints |
| Port local : 8000 | Docker : 8001 |
+-----+
| SSL requis - psycopg2 |
| Port 6543 (connection pooler) |
v
+-----+
| Base de données PostgreSQL |
| Hébergée sur Supabase (finale) |
| aws-0-us-west-2.pooler.supabase.com |
+-----+

```

Evolution de l'infrastructure

- Phase 1 - Docker Desktop *(développement local)* :

- * API FastAPI + PostgreSQL 15 Alpine dans des conteneurs Docker
- * Tout tourne en local sur la machine Windows avec Docker Desktop
- * Commande : `cd Backend && docker-compose up -d`
- * Port API : 8001 ? 8000 | Port BDD : 5432
- * Idéal pour développer sans connexion internet

- Phase 2 - Render.com *(première BDD en ligne)* :

- * Migration de Docker Desktop vers Render.com pour une BDD accessible en ligne
- * Render.com propose une base PostgreSQL gratuite avec chaque compte
- * Host : `dpg-XXXXXX.oregon-postgres.render.com` (port 5432)
- * Limitation rencontrée : BDD mise en veille après inactivité ? données parfois perdues
- * Raison du passage à Supabase : plus stable, toujours actif

- Phase 3 - Supabase *(solution finale et actuelle)* :

- * Migration depuis Render vers Supabase pour plus de fiabilité
- * PostgreSQL hébergé sur AWS Oregon, jamais en veille
- * Connexion via Connection Pooler (port 6543, SSL obligatoire)
- * Backups automatiques quotidiens inclus

- * Interface web (Table Editor) pour voir et modifier les données facilement
-

Détail des fichiers

database/ - Schémas et données SQL

- schema.sql - A exécuter pour créer la BDD :

- * Table users - Comptes utilisateurs + solde BKN
- * Table transactions - Historique (achat, vente, transfert, réception)
- * Table crypto_transactions - Achats/ventes de cryptomonnaies
- * Table user_settings - Préférences de sécurité (biométrie, 2FA, notifs)
- * Table user_sessions - Sessions actives et appareils connectés
- * 6 index de performance - Sur les colonnes les plus interrogées

- seed.sql - Données de test initiales :

- * 4 utilisateurs (john, jane, bob, alice) - mot de passe : password123
- * 4 transactions BKN de démonstration
- * 3 transactions crypto (bitcoin, ethereum, solana)
- * 3 sessions actives d'exemple
- * Paramètres de sécurité par défaut pour chaque user

deploy/ - Guides de déploiement

- render.md - Historique + guide Render.com :

- * Explication du rôle de Render.com dans le projet (BDD gratuite en ligne)
- * Problèmes rencontrés avec la BDD Render (mise en veille, données perdues)
- * Guide optionnel pour héberger l'API FastAPI sur Render

- supabase.md - Solution finale (base de données) :

- * Création du projet Supabase
- * Récupération des informations de connexion (Connection Pooler port 6543)
- * Exécution du schema.sql via l'éditeur SQL Supabase
- * Pourquoi le port 6543 et pas 5432
- * Procédure de reset complet de la base

env/ - Variables d'environnement

- .env.example - Template à copier dans Backend/.env :

- * Variables de connexion Supabase (DB_HOST, DB_PORT, DB_USER, DB_PASSWORD)
 - * Clé secrète JWT (JWT_SECRET_KEY)
 - * Clé API SendGrid (emails reset mot de passe)
 - * Section commentée pour la configuration Docker locale
-

Docker Desktop - Phase 1 (local)

- Prérequis :

- * Docker Desktop installé et démarré
- * WSL2 activé recommandé sur Windows

- Lancer l'environnement complet :

```
cd Backend
docker-compose up -d
```

- Conteneurs démarrés :

- * bkn_postgres - PostgreSQL 15 Alpine (port 5432)
 - BDD locale, volume persistant : bkn_postgres_data
 - Indépendante de Supabase
- * bkn_api - Python 3.11-slim FastAPI (port 8001 ? 8000)
 - Attend que PostgreSQL soit sain (healthcheck) avant de démarrer
 - Réseau interne : bkn_network (bridge)

- Commandes utiles :

```
docker-compose ps          # Voir l'état des conteneurs
docker-compose logs api    # Logs de l'API
docker-compose down        # Arrêter
docker-compose down -v     # Arrêter + supprimer les volumes (reset BDD)
```

- Scripts Windows disponibles :

- * Backend/lancer_serveur.bat - Lance uniquement le serveur Python FastAPI (python server.py)
- * Backend/docker-start.bat - Lance les conteneurs Docker (docker-compose build + up)

- * Backend/docker-stop-all.bat - Arrête tous les conteneurs Docker
-

Render.com - Phase 2 (1ère BDD en ligne)

- Ce qui a été utilisé :

- * Base PostgreSQL gratuite fournie par Render.com
- * Accessible depuis n'importe où (pas juste en local)
- * Même région Oregon que Supabase - migration facile ensuite

- Limitation rencontrée :

- * Mise en veille de la BDD après période d'inactivité
 - * "Cold start" long (30-50 sec) au réveil
 - * Migration vers Supabase pour plus de stabilité
-

Supabase - Phase 3 (solution finale)

- Connexion (Connection Pooler) :

- * Host : aws-0-us-west-2.pooler.supabase.com
- * Port : 6543 (pooler - mode Transaction)
- * Database : postgres
- * SSL : require (obligatoire)

- Pourquoi Supabase est meilleur que Render pour la BDD ? :

- * Jamais en veille - toujours disponible immédiatement
 - * Connection Pooler (port 6543) = meilleure gestion des connexions simultanées
 - * Interface web pour voir et modifier les données en temps réel
 - * Backups automatiques quotidiens
-

A FAIRE - Tâches restantes

- Priorité haute :

- * [] Déplacer les credentials DB de server.py vers le fichier .env
- * [] Restreindre les CORS en production (pas allow_origins=["*"])
- * [] Implémenter les tokens JWT signés (python-jose est déjà installé)

- Priorité moyenne :

- * [] Ajouter un fichier render.yaml (déploiement Infrastructure as Code)
- * [] Créer un Makefile racine (make start, make stop, make migrate)
- * [] Mettre en place un monitoring uptime (UptimeRobot gratuit)

- Améliorations futures :

- * [] Système de migrations Alembic (évolution du schéma sans reset)
 - * [] Pipeline CI/CD GitHub Actions (lint + tests + deploy auto)
 - * [] Stocker les avatars sur Supabase Storage (au lieu de /avatars/ local)
 - * [] Ajouter des tests unitaires pour les endpoints critiques
-

Points de sécurité à corriger

- Credentials hardcodés dans server.py :

- * Les informations de connexion Supabase sont actuellement écrites directement dans le code source de server.py (lignes 104-108)
- * A déplacer dans le fichier .env et charger avec python-dotenv
- * Risque : exposition des credentials sur GitHub si le fichier est commité

- CORS trop ouvert :

- * allow_origins=["*"] autorise n'importe quel domaine à appeler l'API
- * En production, restreindre aux domaines autorisés uniquement

- Authentification sans JWT signé :

- * Actuellement le login retourne les données brutes de l'utilisateur
 - * Implémenter un vrai token JWT avec python-jose (déjà dans requirements.txt)
 - * Protéger les endpoints sensibles avec un middleware d'authentification
-

Dernière mise à jour : Avril 2026 | Auteur : Patrice Beausoleil